

MEESHO – WOMEN’S FASHION E-COMMERCE ORDER MANAGEMENT SYSTEM

CHAPTER – 5

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

1. Introduction

The Meesho Women’s Fashion E-Commerce Order Management System is designed to manage customer, seller, product, category, order, payment and delivery information efficiently. The system uses related tables to store and organize product and order details. Primary key and foreign key constraints are used to maintain data integrity and establish relationships between different entities.

SQL operations such as INSERT, UPDATE and DELETE are performed to manage product and order information. Reports can be generated using SQL and PL/SQL to analyze products, orders, stock and category-wise information.

2. Objectives

- Design tables for Customer, Seller, Category, Product, Order, Payment and Delivery entities.
- Define primary key and foreign key relationships.
- Store customer, seller, product and category details.
- Manage product price and stock information.
- Store and manage customer orders.
- Maintain payment and delivery details.
- Perform product and order insertion, updating and deletion.
- Generate category-wise and order-wise reports using SQL and PL/SQL.

3. Table Design

3.1 Customer Table

Attribute	Data Type	Key	Description
Customer_ID	NUMBER	Primary Key	Unique customer identifier
Customer_Name	VARCHAR2(100)	-	Customer name
Email	VARCHAR2(100)	Unique	Customer email address
Phone	VARCHAR2(15)	Unique	Customer phone number
Address	VARCHAR2(200)	-	Customer delivery address

3.2 Seller Table

Attribute	Data Type	Key	Description
-----------	-----------	-----	-------------

Seller_ID	NUMBER	Primary Key	Unique seller identifier
Seller_Name	VARCHAR2(100)	-	Seller name
Email	VARCHAR2(100)	Unique	Seller email address
Phone	VARCHAR2(15)	Unique	Seller phone number

3.3 Category Table

Attribute	Data Type	Key	Description
Category_ID	NUMBER	Primary Key	Unique category identifier
Category_Name	VARCHAR2(100)	-	Product category name

3.4 Product Table

Attribute	Data Type	Key	Description
Product_ID	NUMBER	Primary Key	Unique product identifier
Product_Name	VARCHAR2(100)	-	Product name
Price	NUMBER(10,2)	-	Product price
Stock	NUMBER	-	Available quantity
Seller_ID	NUMBER	Foreign Key	References Seller(Seller_ID)
Category_ID	NUMBER	Foreign Key	References Category(Category_ID)

3.5 Order Table

Attribute	Data Type	Key	Description
Order_ID	NUMBER	Primary Key	Unique order identifier
Customer_ID	NUMBER	Foreign Key	References Customer(Customer_ID)
Product_ID	NUMBER	Foreign Key	References Product(Product_ID)
Quantity	NUMBER	-	Quantity ordered
Order_Date	DATE	-	Date of order
Order_Status	VARCHAR2(50)	-	Current order status

3.6 Payment Table

Attribute	Data Type	Key	Description
Payment_ID	NUMBER	Primary Key	Unique payment identifier
Order_ID	NUMBER	Foreign Key	References Order(Order_ID)
Payment_Method	VARCHAR2(50)	-	Mode of payment
Payment_Status	VARCHAR2(50)	-	Payment status

Amount	NUMBER(10,2)	-	Payment amount
--------	--------------	---	----------------

3.7 Delivery Table

Attribute	Data Type	Key	Description
Delivery_ID	NUMBER	Primary Key	Unique delivery identifier
Order_ID	NUMBER	Foreign Key	References Order(Order_ID)
Delivery_Date	DATE	-	Expected delivery date
Delivery_Status	VARCHAR2(50)	-	Current delivery status

4. Relationship

The Meesho database contains relationships between customers, sellers, categories, products, orders, payments and deliveries.

- One customer can place many orders.
- One seller can sell many products.
- One category can contain many products.
- One product can appear in many orders.
- Each order is associated with one payment.
- Each order is associated with one delivery.

CUSTOMER (1) ————— (M) ORDER

SELLER (1) ————— (M) PRODUCT

CATEGORY (1) ————— (M) PRODUCT

PRODUCT (1) ————— (M) ORDER

ORDER (1) ————— (1) PAYMENT

ORDER (1) ————— (1) DELIVERY

5. SQL Table Creation

```
CREATE TABLE Customer (
    Customer_ID NUMBER PRIMARY KEY,
    Customer_Name VARCHAR2(100),
    Email VARCHAR2(100) UNIQUE,
    Phone VARCHAR2(15) UNIQUE,
    Address VARCHAR2(200)
);
```

```
CREATE TABLE Seller (
    Seller_ID NUMBER PRIMARY KEY,
    Seller_Name VARCHAR2(100),
```

```
Email VARCHAR2(100) UNIQUE,  
Phone VARCHAR2(15) UNIQUE  
);
```

```
CREATE TABLE Category (  
    Category_ID NUMBER PRIMARY KEY,  
    Category_Name VARCHAR2(100)  
);
```

```
CREATE TABLE Product (  
    Product_ID NUMBER PRIMARY KEY,  
    Product_Name VARCHAR2(100),  
    Price NUMBER(10,2),  
    Stock NUMBER,  
    Seller_ID NUMBER,  
    Category_ID NUMBER,  
    FOREIGN KEY (Seller_ID) REFERENCES Seller(Seller_ID),  
    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)  
);
```

```
CREATE TABLE Orders (  
    Order_ID NUMBER PRIMARY KEY,  
    Customer_ID NUMBER,  
    Product_ID NUMBER,  
    Quantity NUMBER,  
    Order_Date DATE,  
    Order_Status VARCHAR2(50),  
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
    FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
);
```

```
CREATE TABLE Payment (  
    Payment_ID NUMBER PRIMARY KEY,  
    Order_ID NUMBER UNIQUE,  
    Payment_Method VARCHAR2(50),  
    Payment_Status VARCHAR2(50),  
    Amount NUMBER(10,2),  
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
);
```

```
CREATE TABLE Delivery (  
    Delivery_ID NUMBER PRIMARY KEY,  
    Order_ID NUMBER UNIQUE,
```

```
Delivery_Date DATE,  
Delivery_Status VARCHAR2(50),  
FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
);
```

6. Insert Records

Sample records can be inserted into the Customer, Seller, Category, Product, Orders, Payment and Delivery tables using the SQL INSERT command.

```
INSERT INTO Category VALUES (1, 'Sarees');  
INSERT INTO Category VALUES (2, 'Kurtis');  
INSERT INTO Category VALUES (3, 'Western Wear');
```

```
INSERT INTO Seller VALUES  
(101, 'Fashion Hub', 'fashionhub@gmail.com', '9876543210');
```

```
INSERT INTO Product VALUES  
(1001, 'Cotton Saree', 699.00, 50, 101, 1);
```

```
INSERT INTO Product VALUES  
(1002, 'Printed Kurti', 499.00, 40, 101, 2);
```

```
INSERT INTO Customer VALUES  
(201, 'Anitha', 'anitha@gmail.com', '9876501234', 'Chennai');
```

```
INSERT INTO Orders VALUES  
(301, 201, 1001, 2, SYSDATE, 'Placed');
```

7. Product and Order Insertion, Updating and Deletion

Product Insertion

```
INSERT INTO Product VALUES (1003, 'Floral Dress', 799.00, 30, 101, 3);
```

Product Updating

```
UPDATE Product SET Price = 749 WHERE Product_ID = 1003;
```

Product Deletion

```
DELETE FROM Product WHERE Product_ID = 1003;
```

Order Updating

```
UPDATE Orders SET Order_Status = 'Delivered' WHERE Order_ID = 301;
```

These INSERT, UPDATE and DELETE operations help in maintaining accurate and up-to-date product and order information.

8. Category-wise Product Report (SQL)

```
SELECT C.Category_Name,  
       COUNT(P.Product_ID) AS Total_Products,  
       SUM(P.Stock) AS Total_Stock  
FROM Category C  
LEFT JOIN Product P  
ON C.Category_ID = P.Category_ID  
GROUP BY C.Category_Name;
```

9. PL/SQL Program – Category-wise Report

```
DECLARE  
    CURSOR category_cursor IS  
        SELECT C.Category_Name,  
               COUNT(P.Product_ID) AS Total_Products,  
               NVL(SUM(P.Stock),0) AS Total_Stock  
        FROM Category C  
        LEFT JOIN Product P  
        ON C.Category_ID = P.Category_ID  
        GROUP BY C.Category_Name;  
BEGIN  
    FOR rec IN category_cursor LOOP  
        DBMS_OUTPUT.PUT_LINE(  
            'Category: ' || rec.Category_Name ||  
            ' | Products: ' || rec.Total_Products ||  
            ' | Stock: ' || rec.Total_Stock  
        );  
    END LOOP;  
END;  
/
```

10. PL/SQL Program – Product Price Update

```
DECLARE  
    v_product_id NUMBER := 1001;  
    v_new_price NUMBER := 749;  
BEGIN  
    UPDATE Product  
    SET Price = v_new_price  
    WHERE Product_ID = v_product_id;  
  
    DBMS_OUTPUT.PUT_LINE(  
        'Product price updated successfully.'  
    );
```

```
COMMIT;  
END;  
/
```

11. Expected Category-wise Report

Category	Total Products	Total Stock
Sarees	1	50
Kurtis	1	40
Western Wear	1	30

The report displays the number of products and total available stock under each category.

12. Conclusion

The Meesho Women's Fashion E-Commerce Order Management System successfully represents the relationships between customers, sellers, categories, products, orders, payments and deliveries. Primary key and foreign key constraints help maintain data integrity and establish relationships between the tables.

INSERT, UPDATE and DELETE operations support effective product and order management. SQL reports provide useful information about products and stock, while PL/SQL programs demonstrate procedural database programming for report generation and product price updates.